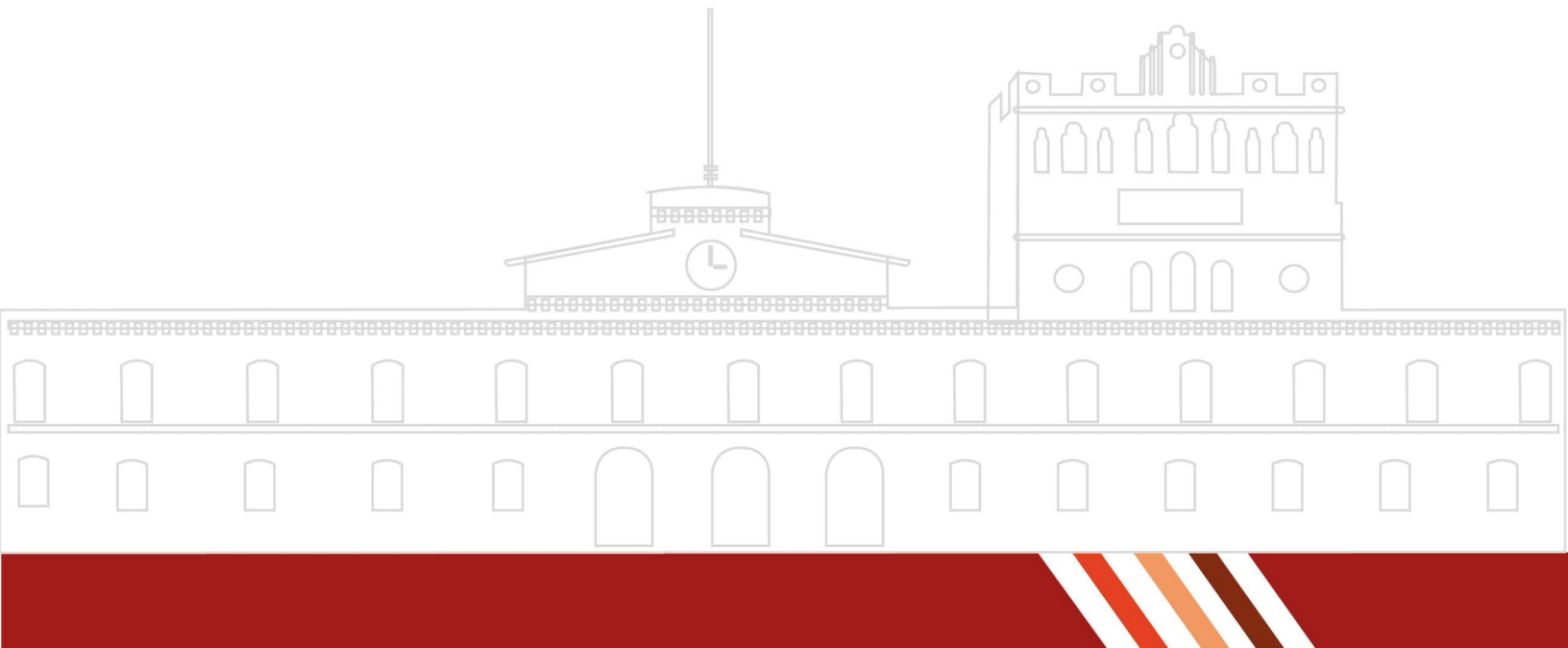


REPORTE DE PRÁCTICA NO. 6

NOMBRE DE LA PRÁCTICA: Análisis sintáctico de expresiones aritméticas

ALUMNOS: Garcia Colmenares Andres
Arana Juarez Raul
Flores Guzman Oliver Alain

Dr. Eduardo Cornejo-Velázquez



1. Introducción

Sintaxis y Gramáticas en los Lenguajes de Programación

Todo lenguaje de programación está regido por un conjunto de reglas que determinan la estructura sintáctica de los programas bien formados. La descripción de la sintaxis de estas construcciones puede realizarse mediante Gramáticas Libres de Contexto o utilizando la Notación BNF (Backus-Naur Form). Estas opciones ofrecen ventajas para los diseñadores de lenguajes, así como a los desarrolladores de compiladores, ya que constituyen especificaciones sintácticas claras y precisas.

Una gramática permite la generación automática de analizadores sintácticos, contribuyendo a la detección de ambigüedades durante el proceso de construcción. Al proporcionar una estructura formal al lenguaje de programación, facilita la generación de código y la identificación de errores. Además, la descripción de un lenguaje mediante una gramática simplifica su ampliación o modificación futura. donde:

Analizador Sintactico

El analizador sintáctico constituye una etapa esencial en el proceso de compilación, encargándose de verificar que el texto de entrada cumpla con las reglas definidas por la gramática correspondiente. Si el programa de entrada es válido, el analizador genera un árbol sintáctico que representa su estructura. En teoría, la salida del analizador consiste en una representación formal del árbol sintáctico basada en la secuencia de tokens proporcionada por el analizador léxico.

En la práctica, el analizador sintáctico suele realizar tareas adicionales, como:

- Acceso a la tabla de símbolos para asistir en las funciones del analizador semántico.
- Verificación de tipos, propia del análisis semántico.
- Generación de código intermedio.
- Manejo y generación de mensajes de error.

Manejo de Errores Sintácticos

El diseño de un compilador sería significativamente más sencillo si solo tuviera que procesar programas correctos. Sin embargo, en la práctica, los programadores suelen cometer errores, por lo que un buen compilador debe ser capaz de identificar, localizar y comunicar estos errores de manera efectiva. Incorporar estrategias de manejo de errores desde las etapas iniciales del desarrollo de un compilador no solo simplifica su estructura, sino que también mejora su capacidad para responder a los errores detectados.

Los errores que pueden surgir durante el proceso de programación incluyen:

- Errores léxicos. Escribir incorrectamente un identificador, palabra clave u operador.
- Errores sintácticos. Derivados de expresiones mal formadas, como paréntesis desbalanceados o operadores mal ubicados.
- Errores semánticos. Aplicar un operador a un operando incompatible.
- Errores lógicos. Códigos con funciones recursivas infinitas.

El manejo de errores sintácticos es especialmente complejo en la creación de compiladores. Un buen analizador sintáctico debe cumplir con los siguientes objetivos:

- Reportar los errores de forma clara y precisa.
- Especificar el tipo de error y su ubicación.
- Recuperarse del error para continuar con el análisis del programa.
- Minimizar el impacto en el rendimiento de la compilación.

Por lo tanto, un compilador robusto debe diseñarse considerando los errores potenciales, lo que no solo mejora su funcionalidad, sino que también garantiza una respuesta más eficiente ante los errores detectados.

2. Marco Teórico

Lenguaje SQL y su Importancia

El lenguaje SQL (Structured Query Language) es el principal lenguaje utilizado para la gestión y manipulación de bases de datos relacionales. Su desarrollo fue con el propósito de permitir la creación, consulta, modificación y eliminación de información almacenada en tablas estructuradas. SQL se utiliza en sistemas gestores de bases de datos como MySQL o PostgreSQL por su capacidad para interactuar con grandes volúmenes de datos de manera eficiente.

Sus instrucciones se clasifican en diferentes categorías según su función. Dentro de estas categorías existen las sentencias de manipulación de datos, entre las cuales se encuentran **SELECT**, **INSERT**, **UPDATE** y **DELETE**. Estas instrucciones son las que permiten consultar registros, insertar nuevos datos, modificar información existente y eliminar registros almacenados en las tablas de una base de datos.

2.1 Gramáticas Libres de Contexto aplicadas a SQL

Las Gramáticas Libres de Contexto son herramientas formales utilizadas para describir la estructura sintáctica de un lenguaje. En el caso de SQL, las gramáticas permiten definir cómo deben construirse las consultas válidas mediante reglas de producción.

Una gramática está compuesta por:

- Un conjunto de símbolos terminales.
- Un conjunto de símbolos no terminales.
- Un símbolo inicial.
- Un conjunto de reglas de producción.

Por ejemplo, una regla para una instrucción **SELECT** se le puede expresar de la siguiente manera:

```
SELECT -> SELECT lista_columnas FROM tabla
```

Estas reglas permiten construir analizadores sintácticos capaces de validar expresiones SQL y detectar errores en la estructura de las consultas.

2.2 Analizador Léxico

El analizador léxico es la primera etapa del proceso de compilación o interpretación. Su función principal es leer el texto de entrada y convertirlo en una secuencia de tokens que puedan ser utilizados por el analizador sintáctico.

En una consulta SQL, los tokens pueden corresponder a:

- Palabras reservadas (**SELECT**, **INSERT**, **UPDATE**, **DELETE**).
- Identificadores (nombres de tablas y columnas).
- Operadores (**=**, **>**, **<**).
- Delimitadores (comas, paréntesis, punto y coma).
- Constantes numéricas o cadenas de texto.

Por ejemplo, la instrucción:

```
SELECT nombre FROM alumnos;
```

puede dividirse en los siguientes tokens:

```
SELECT | ID | FROM | ID | ;
```

El analizador léxico suele implementarse mediante herramientas como Flex, las cuales permiten definir expresiones regulares para identificar cada tipo de token.

2.3 Analizador Sintáctico

El analizador sintáctico recibe una secuencia de tokens que es generada por el analizador léxico y después verifica si estos cumplen con las reglas gramaticales establecidas en sus reglas, en este caso el lenguaje SQL.

Su objetivo principal es determinar si una consulta está correctamente estructurada. Si la expresión es válida, el analizador contruye un árbol sintáctico el cual represente la jerarquía de la instrucción. Si no la es, se generan mensajes de error indicando la naturaleza del problema encontrado.

Un análisis sintáctico puede implementarse utilizando analizadores descendentes o ascendentes. Herramientas como Yacc o Bison permiten la generación de analizadores sintácticos a partir de una gramática definida por el desarrollador.

2.4 Sentencia SELECT

La instrucción **SELECT** es una de las sentencias más importantes de SQL, ya que permite recuperar información almacenada en las tablas de una base de datos.

Su estructura básica es la siguiente:

```
SELECT columnas FROM tabla;
```

También es posible incluir cláusulas adicionales como:

- **WHERE:** filtra registros.
- **ORDER BY:** ordena resultados.
- **GROUP BY:** agrupa registros.
- **HAVING:** filtra grupos.

Ejemplo:

```
SELECT nombre, edad FROM alumnos WHERE edad > 18;
```

El analizador sintáctico debe validar la correcta posición y uso de cada cláusula dentro de la consulta.

2.5 Sentencia INSERT

La instrucción **INSERT** se utiliza para agregar nuevos registros a una tabla. Su sintaxis general es:

```
INSERT INTO tabla (campos) VALUES (valores);
```

Ejemplo:

```
INSERT INTO alumnos (nombre, edad) VALUES ('Ana', 20);
```

El analizador es el encargado de comprobar que la cantidad de campos de la sentencia coincida con la cantidad de valores proporcionados y que su estructura sea correcta.

2.6 Sentencia UPDATE

La instrucción UPDATE permite modificar datos existentes dentro de una tabla. Su estructura básica es:

```
UPDATE tabla SET campo = valor WHERE condicion;
```

Ejemplo:

```
UPDATE alumnos SET edad = 21 WHERE nombre = 'Ana';
```

Durante el análisis sintáctico se verifica la correcta utilización de la cláusula SET, así como la presencia adecuada de condiciones dentro de WHERE.

2.7 Sentencia DELETE

La instrucción DELETE se emplea para eliminar registros de una tabla. Su sintaxis básica es:

```
DELETE FROM tabla WHERE condicion;
```

Ejemplo:

```
DELETE FROM alumnos WHERE edad < 18;
```

El analizador sintáctico debe validar la estructura de la sentencia y garantizar que las cláusulas estén colocadas correctamente.

2.8 Manejo de Errores Sintácticos en SQL

Durante el análisis de consultas SQL pueden presentarse errores sintácticos derivados de expresiones mal formadas. Algunos ejemplos comunes son:

- Omisión de palabras reservadas.
- Uso incorrecto de comas o paréntesis.
- Cláusulas mal posicionadas.
- Falta de punto y coma al finalizar la instrucción.

Un analizador sintáctico robusto debe detectar estos errores y proporcionar mensajes claros que permitan al usuario identificar y corregir el problema rápidamente.

2.9 Herramientas Flex y Bison

Flex y Bison son algunas herramientas ampliamente utilizadas para la construcción de compiladores y analizadores.

Flex es utilizado para generar analizadores léxicos a partir de expresiones regulares, mientras que **Bison** es el que permite construir analizadores sintácticos basados en gramáticas libres de contexto.

La integración de ambas herramientas facilita el desarrollo de sistemas capaces de reconocer y validar expresiones SQL, automatizando gran parte del proceso de análisis sintáctico.

- Flex identifica los tokens para su entrada.
- Bison valida la estructura gramatical con los tokens.

3. Herramientas empleadas

Para esta practica se usaron dos herramientas principales.

1. Lex (o Flex): Genera un analizador léxico en C a partir de un archivo con definiciones de expresiones regulares y acciones (retornar tokens).
2. Yacc (o Bison): Genera un analizador sintáctico en C a partir de una gramática libre de contexto con acciones asociadas (validación, construcción de AST, mensajes de error).

Ambas herramientas siguen el modelo de análisis LALR(1) (Look-Ahead Left-to-Right, con 1 token de anticipación), muy eficiente para la mayoría de los lenguajes de programación.

4. Desarrollo

Base del proyecto

Para esta práctica se partió de los códigos que se desarrollaron anteriormente por el equipo, que usaba una estructura inicial para el análisis léxico y sintáctico.

A partir de este archivo base se realizaron modificaciones y ampliaciones necesarias para poder adaptar el analizador al reconocimiento de instrucciones SQL, específicamente las sentencias:

- SELECT
- INSERT
- UPDATE
- DELETE

Su objetivo principal fue la implementación de un analizador sintáctico capaz de validar la estructura gramatical de estas instrucciones utilizando las herramientas Lex y Yacc.

Desarrollo del programa en Lex

El programa en Lex tenía la función de identificar los distintos componentes léxicos presentes en las instrucciones SQL a partir de expresiones regulares. Para ello se definieron expresiones regulares capaces de reconocer los siguientes elementos:

- Palabras reservadas de SQL (SELECT, FROM, WHERE, INSERT, UPDATE, DELETE, etc.).
- Identificadores (nombres de tablas y columnas).
- Operadores relacionales (=, >, <).
- Separadores (comas, paréntesis y punto y coma).
- Valores numéricos.
- Cadenas de texto.

El analizador léxico transforma la entrada proporcionada por el usuario en una secuencia de tokens que posteriormente son enviados al analizador sintáctico generado con Yacc.

Código en Lex:

```
1 %option noyywrap
2 %{
3 #include <stdio.h>
4 #include "sql.tab.h"
5 int linea = 1;
6 %}
7
8 LETRA      [a-zA-Z_]
9 DIGITO     [0-9]
10 ID         {LETRA}({LETRA}|{DIGITO})*
11 ESPACIO    [ \t\r]+
12
13 %%
14
15 "SELECT" | "select"          { return SELECT; }
16 "INSERT" | "insert"         { return INSERT; }
```



```

17 "INTO" | "into"           { return INTO; }
18 "VALUES" | "values"      { return VALUES; }
19 "UPDATE" | "update"      { return UPDATE; }
20 "DELETE" | "delete"      { return DELETE; }
21 "FROM" | "from"          { return FROM; }
22 "WHERE" | "where"        { return WHERE; }
23 "AND" | "and"             { return AND; }
24 "OR" | "or"               { return OR; }
25 "SET" | "set"             { return SET; }
26
27 {ID}                      { yylval = 0; return IDENTIFICADOR; }
28
29 {DIGITO}+                 { yylval = 0; return ENTERO; }
30
31 \'([^\'\\]|\\.)*\'         { yylval = 0; return CADENA; }
32
33 ","                      { return COMA; }
34 ";"                      { return PUNTOYCOMA; }
35 "="                      { return IGUAL; }
36 "<>"                     { return DISTINTO; }
37 "!="                     { return DISTINTO; }
38 "<="                     { return MENORIGUAL; }
39 ">="                     { return MAYORIGUAL; }
40 "<"                      { return MENOR; }
41 ">"                      { return MAYOR; }
42 "("                      { return '('; }
43 ")"                      { return ')'; }
44 "*"                      { return '*'; }
45
46 "--".*                   { /* ignorar */ }
47
48 {ESPACIO}                 { /* ignorar */ }
49
50 \n { linea++; }
51
52 .                         { printf("No reconocido"); }
53
54
55 %%

```

Desarrollo del programa en Yacc

El programa desarrollado en Yacc se encargó de verificar que las instrucciones SQL cumplieran correctamente con las reglas gramaticales establecidas para cada sentencia.

Se definieron producciones sintácticas para validar estructuras correspondientes a las operaciones:

- Consultas con SELECT.
- Inserción de datos mediante INSERT.
- Actualización de registros utilizando UPDATE.
- Eliminación de registros con DELETE.

Además, se agregaron acciones semánticas que permiten indicar si la consulta ingresada es válida o si contiene errores sintácticos.

```
1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int yylex();
6  int yyparse();
7
8  extern FILE *yyin;
9  extern FILE *yyout;
10 extern char *yytext;
11 extern int linea;
12
13 int Errores = 0;
14
15 void yyerror(const char *s)
16 {
17     Errores = 1;
18
19     fprintf(
20         yyout,
21         "[Linea %d] Error cerca de '%s': %s\n",
22         linea,
23         yytext,
24         s
25     );
26 }
27
28 %}
29
30 %token SELECT INSERT UPDATE DELETE
31 %token INTO VALUES SET FROM WHERE AND OR
32 %token IDENTIFICADOR
33 %token ENTERO CADENA
34 %token COMA PUNTOYCOMA
35 %token IGUAL MENOR MAYOR MENORIGUAL MAYORIGUAL DISTINTO
36
37 %left OR
38 %left AND
39 %left IGUAL DISTINTO MENOR MENORIGUAL MAYOR MAYORIGUAL
40
41 %start input
42
```

```

43 %%
44
45 input:
46     /* vac o */
47     | input sentencia
48     ;
49
50 sentencia:
51     select_stmt PUNTOYCOMA
52     {
53         fprintf(yyout,
54             "[Linea %d] Sentencia SELECT valida\n",
55             linea);
56     }
57
58     | insert_stmt PUNTOYCOMA
59     {
60         fprintf(yyout,
61             "[Linea %d] Sentencia INSERT valida\n",
62             linea);
63     }
64
65     | update_stmt PUNTOYCOMA
66     {
67         fprintf(yyout,
68             "[Linea %d] Sentencia UPDATE valida\n",
69             linea);
70     }
71
72     | delete_stmt PUNTOYCOMA
73     {
74         fprintf(yyout,
75             "[Linea %d] Sentencia DELETE valida\n",
76             linea);
77     }
78
79     | error PUNTOYCOMA
80     {
81         fprintf(yyout,
82             "[Linea %d] Sentencia invalida\n",
83             linea);
84
85         yyerrok;
86     }
87 ;
88
89 select_stmt:
90     SELECT select_lista FROM IDENTIFICADOR opt_where
91     ;
92
93 select_lista:
94     '*'
95     | lista_columnas
96     ;
97
98 lista_columnas:
99     IDENTIFICADOR
100     | lista_columnas COMA IDENTIFICADOR
101     ;

```

```

102
103 insert_stmt:
104     INSERT INTO IDENTIFICADOR opt_columnas VALUES '(' lista_valores ')',
105     ;
106
107 opt_columnas:
108     /* vac o */
109     | '(' lista_columnas ')',
110     ;
111
112 lista_valores:
113     valor
114     | lista_valores COMA valor
115     ;
116
117 valor:
118     ENTERO
119     | CADENA
120     ;
121
122 update_stmt:
123     UPDATE IDENTIFICADOR SET lista_asignaciones opt_where
124     ;
125
126 lista_asignaciones:
127     asignacion
128     | lista_asignaciones COMA asignacion
129     ;
130
131 asignacion:
132     IDENTIFICADOR IGUAL valor
133     ;
134
135 delete_stmt:
136     DELETE FROM IDENTIFICADOR opt_where
137     ;
138
139 opt_where:
140     /* vac o */
141     | WHERE condicion
142     ;
143
144 condicion:
145     condicion AND condicion
146     | condicion OR condicion
147     | '(' condicion ')',
148     | comparacion
149     ;
150
151 comparacion:
152     IDENTIFICADOR operador valor
153     ;
154
155 operador:
156     IGUAL
157     | DISTINTO
158     | MENOR
159     | MENORIGUAL
160     | MAYOR

```

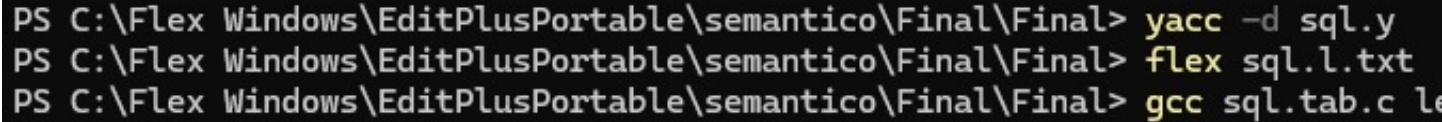
```
161 | MAYORIGUAL
162 ;
163
164 %%
165
166 int main()
167 {
168     FILE *archivo;
169
170     linea = 1;
171
172     yyout = fopen("resultado.txt", "w");
173
174     if (yyout == NULL)
175     {
176         printf("Error al crear resultado.txt\n");
177         return 1;
178     }
179
180     archivo = fopen("entrada.txt", "r");
181
182     if (archivo == NULL)
183     {
184         printf("Error al abrir entrada.txt\n");
185         fclose(yyout);
186         return 1;
187     }
188
189
190     yyin = archivo;
191
192     yyparse();
193
194     printf("Salida escrita en resultado.txt\n");
195     fclose(archivo);
196     fclose(yyout);
197
198     return 0;
199 }
```

Compilación y ejecución de los programas en Lex y Yacc

Para generar el programa ejecutable del analizador sintáctico fue necesario compilar tanto el archivo de Lex como el archivo de Yacc, en la terminal de windows, debido a que necesitan ser compilador por GCC de forma conjunta

El procedimiento realizado fue el siguiente:

1. Como primer paso, se accedió desde la terminal a la carpeta donde se encontraban almacenados los archivos fuente del proyecto, esto fue sencillo debido a que Visual Studio Code, la herramienta en uso, nos lleva a la carpeta base en el inicio de la terminal.
2. Posteriormente se compiló el archivo de Yacc para generar el analizador sintáctico y los archivos auxiliares necesarios.
3. Después se compiló el archivo de Lex para generar el analizador léxico correspondiente.
4. Finalmente, se utilizaron los archivos generados por ambas herramientas para crear el programa ejecutable del analizador sintáctico SQL.



```
PS C:\Flex Windows\EditPlusPortable\semantico\Final\Final> yacc -d sql.y
PS C:\Flex Windows\EditPlusPortable\semantico\Final\Final> flex sql.l.txt
PS C:\Flex Windows\EditPlusPortable\semantico\Final\Final> gcc sql.tab.c le
```

Figure 1: Generación del archivo ejecutable

Pruebas y resultados

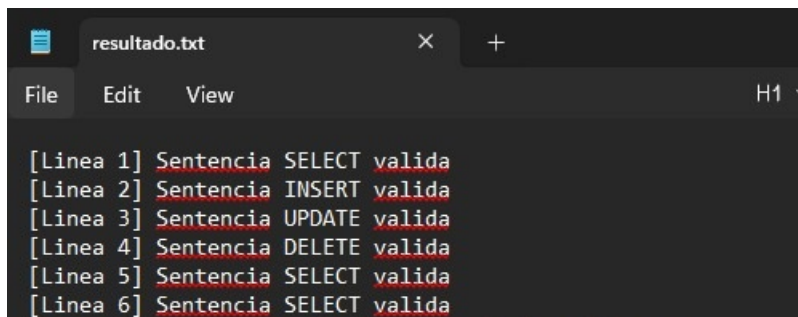
Las pruebas realizadas consistieron en ingresar diferentes instrucciones SQL desde la consola para verificar el correcto funcionamiento del analizador sintáctico.

Se evaluaron tanto casos válidos como inválidos para comprobar la capacidad del sistema de detectar errores sintácticos y aceptar consultas correctamente estructuradas.

Entre las pruebas realizadas se incluyeron:

- Consultas **SELECT** con cláusulas **FROM** y **WHERE**.
- Inserciones mediante **INSERT INTO**.
- Actualizaciones usando **UPDATE SET**.
- Eliminaciones con **DELETE FROM**.

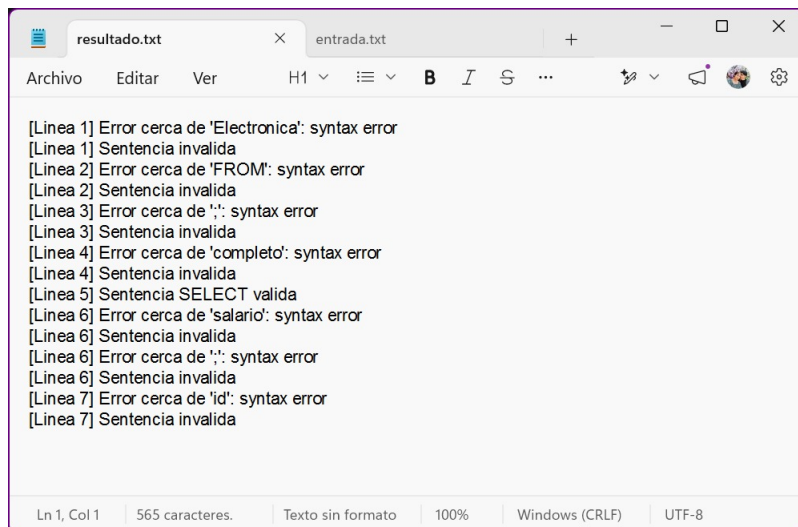
A continuación se muestran algunos ejemplos de casos aceptados y rechazados por el analizador.



A screenshot of a text editor window titled 'resultado.txt'. The editor has a dark background and shows six lines of text. Each line is formatted with a line number in brackets, followed by the word 'Sentencia' in red, the SQL keyword in red, and the word 'valida' in red. The SQL keywords are: SELECT, INSERT, UPDATE, DELETE, SELECT, and SELECT.

```
[Linea 1] Sentencia SELECT valida
[Linea 2] Sentencia INSERT valida
[Linea 3] Sentencia UPDATE valida
[Linea 4] Sentencia DELETE valida
[Linea 5] Sentencia SELECT valida
[Linea 6] Sentencia SELECT valida
```

Figure 2: Casos de aceptación



A screenshot of a text editor window titled 'resultado.txt'. The editor has a light background and shows seven lines of text. Each line is formatted with a line number in brackets, followed by an error message in black. The error messages are: 'Error cerca de 'Electronica': syntax error', 'Sentencia invalida', 'Error cerca de 'FROM': syntax error', 'Sentencia invalida', 'Error cerca de ':' syntax error', 'Sentencia invalida', 'Error cerca de 'completo': syntax error', 'Sentencia invalida', 'Sentencia SELECT valida', 'Error cerca de 'salario': syntax error', 'Sentencia invalida', 'Error cerca de ':' syntax error', 'Sentencia invalida', 'Error cerca de 'id': syntax error', and 'Sentencia invalida'.

```
[Linea 1] Error cerca de 'Electronica': syntax error
[Linea 1] Sentencia invalida
[Linea 2] Error cerca de 'FROM': syntax error
[Linea 2] Sentencia invalida
[Linea 3] Error cerca de ':' syntax error
[Linea 3] Sentencia invalida
[Linea 4] Error cerca de 'completo': syntax error
[Linea 4] Sentencia invalida
[Linea 5] Sentencia SELECT valida
[Linea 6] Error cerca de 'salario': syntax error
[Linea 6] Sentencia invalida
[Linea 6] Error cerca de ':' syntax error
[Linea 6] Sentencia invalida
[Linea 7] Error cerca de 'id': syntax error
[Linea 7] Sentencia invalida
```

Figure 3: Casos de rechazo

5. Cuestionario

1. **¿Cómo se describe la sintaxis de un lenguaje de programación mediante gramáticas (en notación BNF), y qué ventajas ofrecen estas herramientas en el diseño de compiladores?**

La sintaxis es descrita mediante reglas de producción en BNF que define cómo se construyen las expresiones válidas del lenguaje. Estas gramáticas son las que facilitan el diseño de compiladores, la detección de errores y la generación automática de analizadores sintácticos.

2. **¿Qué roles desempeñan Lex y Yacc en el análisis léxico y sintáctico, y cómo se complementan en el desarrollo de un compilador?**

Son lenguajes de programación que facilitan la creación de tokens y árboles semánticos, para la nueva creación de compiladores.

3. **Al implementar un analizador léxico en Lex, ¿qué patrones se deben definir para identificar correctamente los tokens de números y operadores aritméticos?**

Se deben definir las expresiones regulares correctamente para reconocer números enteros o decimales, y también los operadores como suma, resta, multiplicación y división.

4. **Durante la validación de una línea de código como `133 * 25`, ¿cómo identifica errores sintácticos el programa en Yacc y qué estrategias utiliza para recuperarse de ellos?**

Yacc compara la secuencia de tokens con las reglas de la gramática. Si este encuentra una estructura inválida, procede a generar un mensaje de error y el mismo utiliza reglas de recuperación para continuar el análisis.

6. Conclusiones

A lo largo del desarrollo de nuestra práctica se puso en práctica todos los conocimientos que se han adquirido a lo largo del semestre que hemos necesitado del analizador léxico para poner en práctica el análisis sintáctico. Con esto al momento de compilar pudimos resolver los errores que se tenían debido a la sintaxis diferentes. Al final, se logró comprender el funcionamiento de ambos análisis (léxico y sintáctico) y cómo se relacionan entre sí, además de que se comprobó cómo se necesitan y se relacionan el uno del otro para poder funcionar de manera correcta.

Referencias Bibliográficas

References

- [1] d Aho, A. V., Ravi, S. A. V. & Ullman, J. D. **(1998)**. Compiladores: Principios, técnicas y herramientas. Addison Wesley Longman. México.
- [2] Giró, J., Vázquez, J., Meloni, B., Constable, L. **(2015)**. Lenguajes formales y teoría de autómatas. Editorial Alfaomega. Argentina.